START WITH THE WORK

Source walkthrough

The experiment needs a computer. Then it needs a platform.

A robot policy needs simulated data, training and a test run.
A researcher needs GPUs, an environment and repeatable jobs.

O

OSMO

Coordinate containers, data and hardware as a workflow.

S

SkyPilot

Find compute, prepare it and run jobs across supported infrastructure.

First understand the work. Then trace the Pods. Then decide what a metric can mean.

Sources: 1. OSMO purpose 2. SkyPilot purpose

START WITH THE WORK

Source walkthrough

Two products remove different kinds of manual work

OSMO

“Run these connected tasks.”

Pick suitable hardware for each task. Start the ready tasks. Move their outputs. Keep the workflow moving.

`Simulation → training → evaluation`

SkyPilot

“Run this job on suitable compute.”

Choose feasible infrastructure. Provision or reuse a cluster. Prepare the environment. Submit the process.

`Task YAML → compute cluster → job`

> Both run application work. Their logical objects and execution lifetimes differ.

Sources: 1. OSMO source 2. SkyPilot overview

START WITH THE WORK

Source walkthrough

The author and the operator have different inputs

PERSON	OSMO	SKYPILOT
Application author	Images, commands, task dependencies, data and resource needs	Task YAML, resource requirements, files, setup and run commands
Platform operator	Backend clusters, pools, platforms, access and storage	Cloud / Kubernetes access, API server configuration, policy and shared infrastructure
What the author gets back	A workflow run with task progress, logs and artifacts	A cluster and job, or a managed job with progress and logs

Sources: 1. OSMO source 2. Task YAML contract

START WITH THE WORK

Reading guide

Follow the complete path before opening the metrics

01

OSMO

Author a workflow → persist a run → create task Pods → transfer outputs → complete.

02

SkyPilot

Author a task → choose compute → provision Pods → prepare runtime → queue jobs.

03

Karta

Observe Pods → resolve real roots → attach external identity → choose honest aggregation.



Source snapshots: OSMO 19b5170; SkyPilot 358c245. OSMO lab: 6.3.1.3326e42b. The newer OSMO source and the lab are distinct.

Sources: 1. OSMO source 2. SkyPilot source 3. Research audit and version evidence

OSMO · THE AUTHOR

Documented motivating scenario

A robotics workflow connects different machines

Simulation
Generate camera samples on GPU hardware

images + labels →

Training
Learn a policy from those samples

model weights →

Evaluation
Test the policy in simulation or on an edge device

The work is larger than one command. Each stage has prerequisites, hardware needs and artifacts to preserve.

> OSMO lets the author declare this relationship and lets the platform coordinate execution.

Sources: 1. OSMO source 2. OSMO source

Upstream examples, not lab runs

The cookbook shows three different coordination problems

01

Serial processing

A producer writes a file.
A consumer reads it later.

Coordinate dependencies and transfer.

02

Distributed training

Two torchrun tasks use ranks 0 and 1.
The upstream example asks for 8 GPUs per task.

Coordinate addresses, resources and execution.

03

Hardware in the loop

ROS2 discovery, Isaac Lab and a Jetson policy cooperate.

Coordinate unlike processes and hardware.

Upstream examples. The recorded lab used finite CPU tasks, not these GPU or robotics workloads.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

Teaching example, not run

Start with a complete, small CPU workflow

prepare-and-read.yaml

```yaml
workflow:
  name: prepare-and-read
  tasks:
  - name: prepare
    image: ubuntu:22.04
    command: [sh, -c]
    args: ["echo camera-sample > {{output}}/sample.txt"]
  - name: inspect
    image: ubuntu:22.04
    command: [sh, -c]
    args: ["cat {{input:0}}/sample.txt"]
    inputs:
    - task: prepare
```

Input

Two tasks. Each supplies a container image and a command.

Operation

`prepare` writes a file. `inspect` declares that output as its input.

Expected result

```
camera-sample
```

Adapted from the serial tutorial. This exact teaching file was not submitted in the lab.

Sources: 1. OSMO source 2. OSMO source

The input declaration controls both order and bytes

Meaning of the workflow tokens

```
# prepare writes here:
{{output}}/sample.txt

# inspect reads here:
{{input:0}}/sample.txt

# inspect declares the source:
inputs:
- task: prepare
```

prepare

Writes sample.txt into its local output directory

upload, successful completion

↓

Object storage

Retains the produced artifact

download before inspect command

↓

inspect

Reads the first input directory

> These tokens become filesystem paths. They do not name Kubernetes objects.

OSMO · THE AUTHOR

Documented CLI journey

Submit, inspect and follow the run

OSMO CLI · replace the angle-bracket placeholders

```
osmo workflow submit prepare-and-read.yaml --pool <pool>
osmo workflow query <workflow-id>
osmo workflow logs <workflow-id>
```

Submit to a configured pool

The returned workflow ID identifies the submitted run. Use that ID to inspect progress and logs.

Keep the task specification portable

The operator exposes pools and hardware platforms. The workflow chooses the pool and declares what its tasks need.

Commands describe the documented workflow. They are not a transcript of the teaching example running.

Sources: 1. OSMO source 2. OSMO source

OSMO · THE AUTHOR

Source walkthrough

Pool, platform and backend answer different questions

TERM	THE QUESTION	FOR ONE SUBMITTED WORKFLOW
Pool	Which shared allocation can I use?	The selected pool supplies the backend.
Platform	Which configured hardware does this task need?	Tasks can request different hardware types.
Backend	Which Kubernetes environment executes the run?	The parser assigns one backend to every task.

One OSMO installation can register several backends

Backend A

Selected for this workflow · GPU + ARM platforms

Backend B

Available to another pool or workflow

> Multiple registered clusters does not mean one workflow spans those clusters.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Separate the decisions, execution and artifacts

Control plane

Core API
Validate submissions and expose workflow APIs

Worker + Redis
Queue work and advance ready groups

PostgreSQL
Persist workflow, group and task state

Agent · Logger · Router
Backend bridge · runtime records · interactive connections

prepared specs



WebSocket



observations

Compute backend

Backend worker
Create and clean up Kubernetes resources

Kubernetes + KAI
Schedule task Pods as groups

Task Pod runtimes
Download → execute → upload

Backend listener
Watch Pods, nodes and events

Object storage Artifacts move between storage and task runtimes. Workflow records stay in PostgreSQL.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

A request becomes a persistent run before it becomes a Pod

CLI or UI
Submit YAML to the selected pool

HTTP request →

Core API
Render, parse and validate the workflow

identity + queued work →

Database + Redis
Initial records and SubmitWorkflow job

Persistent identity
One submitted workflow receives workflow, group and task identities. The workflow is an OSMO model stored outside Kubernetes.

Deferred orchestration
The API queues work. A control-plane worker later consumes it and advances the dependency graph.

> There is no Workflow CRD created by this submission path.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Only ready groups enter the execution queue

Workflow prepare-and-read

prepare group

No prerequisites · enqueue CreateGroup

successful output dependency



inspect group

Waiting · no task Pod needed yet

The author wrote tasks

OSMO normalizes each top-level task into a one-task group.

The worker tracks dependencies

SubmitWorkflow persists group relationships and queues groups whose prerequisites are satisfied.

A logical task can exist in the workflow while no corresponding Pod exists.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

The agent bridges queued work into the backend

Control plane

Backend queue

CreateGroup is ready

consume + prepare

Agent service

Generate Kubernetes specifications

Authenticated WebSocket

prepared job payload

Compute backend

Backend worker

Receive the job using its registered connection

Kubernetes API calls

Namespace resources

Create the generated resource specifications

> The backend worker creates resources. The backend listener reports observations.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source 4. OSMO source

OSMO · INSIDE THE PLATFORM

Upstream GPU example, not a lab run

A group is for processes that cooperate

Upstream torchrun group: training

master task

Rank 0 · lead: true · 8 GPUs

worker task

Rank 1 · 8 GPUs

Both use {{host:master}} for the rendezvous address.

Shared execution

The group coordinates resource allocation and task startup. The processes can exchange data over the network.

Distinct tasks

A group can contain different images, commands and hardware. The lead task also influences sibling completion and restart behavior.

> A group is not automatically a homogeneous replica set.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

Source walkthrough

Gang scheduling and command startup are separate gates

1 Can the group get resources?

OSMO creates a KAI PodGroup. For a simple two-task group, `minMember: 2`. Each Pod references the group by annotation.

KAI scheduler · placement

2 Are the runtimes ready?

After containers start and inputs download, task runtimes can wait at a barrier. The logger tracks arrivals in Redis and releases them.

OSMO runtime + logger · execution

> Coordinated scheduling does not promise identical wall-clock command start times.

Sources: 1. OSMO source 2. Active source factory accepts KAI only 3. OSMO source 4. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Open one task Pod

One Kubernetes Pod · restartPolicy: Never

INITIALIZATION

Init container

Prepares the runtime.

APPLICATION CONTAINER

User runtime + command

Runs `cat`, training or simulation.
Relays output and configured checkpoints.

Unix socket
↔

CONTROL CONTAINER

OSMO control runtime

Download inputs, coordinate startup,
upload outputs and report progress.

A task is not three separate Pods. Container-level usage needs an explicit application-container selection.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Follow the file separately from the status

prepare runtime
Writes sample.txt locally

upload bytes →

Object storage
Intermediate artifact

download bytes →

inspect runtime
Reads sample.txt locally

prepare completion + runtime timing ⇢ **Control plane records** ⇠ inspect progress + logs

Data path

The runtimes transfer files to and from object storage. PostgreSQL is not the storage location for camera images or model weights.

Control path

Status reports let OSMO release downstream work and report workflow progress. They do not carry the artifact itself.

Sources: 1. OSMO source 2. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Running includes more than the user command

OSMO RUNNING phase

Download
Acquire task input files

ready →

Execute
Run the application command

finished →

Upload
Preserve task outputs

Low accelerator activity can be consistent with transfer work or coordination inside a running task.

> Pod phase, workflow phase and application utilization answer different questions.

Sources: 1. OSMO source 2. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Two observation paths close the control loop

PRODUCER	PATH	WHAT THE CONTROL PLANE LEARNS
Kubernetes	Backend listener → agent	Pod scheduling and status, node changes, events and heartbeats
Task runtime	Control runtime → logger	Logs, execution timing, input/output timing and barrier arrivals
Group update job	Dependency state → ready-group queue	A successful group can release downstream work; failure can block it

> The listener and logger observe different layers. Neither signal alone describes the whole workflow.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source 4. OSMO source

OSMO · INSIDE THE PLATFORM

Interactive source walkthrough

Walk the submission from YAML to printed output

01	**Submit the workflow**	The CLI sends prepare-and-read.yaml to the selected pool. No task Pod is required to accept the request.
02	**Persist the run and queue ready work**	The core API and worker establish workflow, group and task records. prepare is ready; inspect waits for its dependency.
03	**Prepare resources and cross the backend boundary**	The agent generates Kubernetes specifications. The backend worker creates the ready group resources through the Kubernetes API.
04	**Initialize and execute prepare**	Kubernetes starts the containers. The OSMO runtimes coordinate startup, then the user command writes camera-sample into sample.txt.
05	**Preserve the output and release inspect**	The prepare runtime uploads its output. Successful group completion releases inspect. Its runtime downloads sample.txt into the first input directory.
06	**Print the result and complete**	inspect prints camera-sample. Runtime and Kubernetes observations update the run. Resource cleanup happens separately from the terminal workflow state.

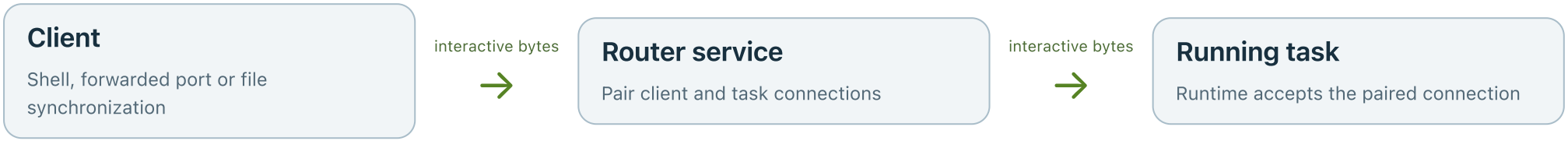
A source-backed simulation of the teaching example. Use the buttons to pause at each component boundary.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source 4. OSMO source

OSMO · INSIDE THE PLATFORM

Source walkthrough

Interactive development uses a separate connection



The core API exposes the requested action. The router relays traffic to an already running task.

> The router is not the workflow queue, and it is not the ROS2 discovery task in the HIL example.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

OSMO · EXECUTION IDENTITY

Source walkthrough

Failure can create a new attempt, not a new logical task

Attempt 0

Workflow W · task T

retry_id = 0

Pod UID A

permitted retry



Attempt 1

Workflow W · task T

retry_id = 1

Pod UID B

Policy decides whether to retry

Exit actions, retry limits and scheduler support matter. A failure does not imply automatic recovery.

Cleanup is a separate operation

OSMO queues cleanup of generated resources. A terminal workflow record and Pod deletion are not one atomic event.

Retry identity is source-verified. Captured lab attempts all had retry_id 0.

Sources: 1. OSMO source 2. OSMO source 3. OSMO source

Source walkthrough

The logical hierarchy and Kubernetes objects are different views

OSMO model

Workflow UUID

One submitted run

contains

Group UUID

Cooperating tasks

contains

Task UUID + retry ID

Logical task and attempt

Kubernetes backend

KAI PodGroup

Coordinates scheduling for the group

Pods reference it through pod-group-name annotations

Task Pod A

Its own Kubernetes UID

Task Pod B

Its own Kubernetes UID

The captured task Pods have empty ownerReferences. No ownership arrow connects sibling Pods.

Sources: 1. OSMO source 2. OSMO source 3. Recorded research evidence

Source walkthrough

The Pod carries several useful identities

OBSERVED FIELD	SCOPE	USE
osmo.workflow_uuid	Submitted run	Collect tasks from the same workflow
osmo.group_uuid	Group within that run	Identify cooperating tasks
osmo.task_uuid / osmo.task_name	Logical task	Distinguish tasks within the workflow
osmo.retry_id	Attempt number	Separate retries of a task
metadata.uid	One Kubernetes Pod incarnation	Join observations without crossing Pod replacement

> Scope external identity by platform installation. Preserve cluster, namespace and Pod UID for Kubernetes observations.

Sources: 1. OSMO source 2. OSMO source

OSMO · EXECUTION IDENTITY

Recorded lab

The lab captured identity, not robotics performance

Sanitized captured Pod · selected fields

```
kind: Pod
metadata:
  namespace: test-ref
  ownerReferences: []
  labels:
    osmo.workflow_id: karta-ref-single-4
    osmo.workflow_uuid: df8c436a2ee642488e061aef2fc5e1f4
    osmo.task_name: worker
    osmo.retry_id: "0"
```

Completed

A finite single task and a two-task CPU group.

Observed

Actual Pod UIDs, empty owner references, shared workflow/group labels and eventual cleanup.

Not exercised

GPU training, robotics performance and live retry behavior.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. Recorded research evidence

OSMO · EXECUTION IDENTITY

Source behavior + recorded identity evidence

Now the useful metrics question is precise

Which Pods executed workflow W,
which task attempt did each represent,
and what resources did they consume?

Kubernetes fact		OSMO fact
Pod UID and real root	+	**Workflow, group, task, attempt**

> The metric can carry both facts without turning a database workflow into a Kubernetes owner.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. OSMO source

SKYPILOT · THE AUTHOR

Source walkthrough

The program is ready. Finding usable compute is another job.

S

One task

Code + environment
+ resource requirements

allowed infrastructure →

- Kubernetes contexts
- Cloud accounts
- Available compute choices

SkyPilot gives an AI developer one task format and interface for choosing compute, preparing it and running the program.

> Infrastructure teams can expose a shared API server. An individual can use the automatically started local server.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · THE AUTHOR

Source walkthrough

Choose reusable compute or a managed run

Interactive iteration

launch + exec

Create or reuse a named cluster. Edit the program and submit more jobs to that compute.

Keep cpu-lab available for experiments.

Unattended execution

jobs launch

Let SkyPilot manage provisioning, monitoring, supported recovery and compute cleanup.

Run cpu-batch without managing its cluster by hand.

The simple managed example uses dedicated temporary compute. Pools and job groups are additional execution models.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · THE AUTHOR

Teaching example, not run

A small task describes compute and work together

cpu-task.yaml

```yaml
name: cpu-check
resources:
  infra: k8s
  cpus: 2
  memory: 4
num_nodes: 1
workdir: .
setup: |
  python3 --version
run: |
  python3 - <<'PYTASK'
  total = sum((i % 100) ** 2 for i in range(5_000_000))
  print(f"squared_feature_sum={total}")
  PYTASK
```

Compute

One node. Two CPUs and 4 GiB memory. Eligible Kubernetes contexts.

Environment

Sync this example directory and check Python during setup.

Work

Run a standard-library calculation and print its result.

Use a dedicated example directory for `workdir: .`. This file is source-reviewed, not cluster-tested.

Sources: 1. SkyPilot source 2. SkyPilot source

SKYPILOT · THE AUTHOR

Teaching example + locally checked arithmetic

Launch once, then submit another execution

SkyPilot CLI · job ID comes from queue

```
sky launch -c cpu-lab cpu-task.yaml
sky exec cpu-lab cpu-task.yaml
sky queue cpu-lab
sky logs cpu-lab <job-id-from-queue>
sky down cpu-lab
```

Expected application output

```
squared_feature_sum=16417500000
```

The arithmetic was checked locally. This is not a captured SkyPilot log.

Two executions, reusable compute

`launch` submits the first job. `exec` submits another to the existing cluster. `down` ends the compute lifetime.

> A successful task parse or Pod creation is not evidence that the application ran.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · THE AUTHOR

Source walkthrough

Task, compute cluster and job are different objects

CONCEPT	IN THIS EXAMPLE	LIFETIME
Task	`cpu-task.yaml`	A reusable specification of resources, files, setup and run
SkyPilot cluster	`cpu-lab`	Compute that can survive several job executions
Cluster job	One ID from `sky queue cpu-lab`	One submitted execution and its status
Kubernetes cluster	The infrastructure hosting the compute Pods	The environment on which SkyPilot provisions

> Running the same task twice does not require two SkyPilot clusters or two Pods.

Sources: 1. SkyPilot source 2. SkyPilot source

Source walkthrough

launch and exec run different preparation steps

OPERATION	LAUNCH	EXEC
Provision or reuse compute	Yes	Use existing compute
Synchronize workdir	Yes	Yes
Synchronize file mounts	Yes in the normal path	Skipped
Run setup	Yes unless skip options apply	Skipped
Submit run command	Yes	Yes

> Editing a dependency installation in setup requires launch to apply it. Editing program files in workdir can be followed by exec.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Source walkthrough

The API server orchestrates; compute runs the program

Client and control plane

CLI / SDK

Task + requested operation

request →

API server

Track the long-running operation

Optimizer

Choose an allowed execution plan

Provisioner

Create or reuse provider compute

Optimizer and provisioner are Python responsibilities, not necessarily separate services.

provider API → Kubernetes scheduling ↓

SkyPilot compute cluster · hosted as Kubernetes Pods

Head Pod

skylet + job queue + Ray + user processes

Worker Pods, if requested

Ray + user processes

Peer compute Pods participate in one runtime. This is not a Pod ownership chain.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source 4. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Source walkthrough

A launch is a long-running API request

Developer terminal

sky launch sends the task

/launch →

API server

Schedule execution.launch as a request

request execution →

Orchestration path

Choose, provision, prepare and submit

A stable entry point

The request continues beyond one synchronous CLI call. A remote server provides shared management for users, clusters and jobs.

Local is also possible

An individual CLI can start a local API server. The deployment mode matters later for managed-job controller placement.

Sources: 1. SkyPilot source 2. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Source walkthrough

Three decisions happen at three different levels

Allowed infrastructure	**SkyPilot optimizer**	Choose a feasible execution plan. The default objective is cost; explicit requirements constrain the choices.
Physical Kubernetes nodes	**Kubernetes scheduler**	Place the requested compute Pods on nodes that can satisfy their requests.
Resources inside compute	**SkyPilot queue + Ray**	Start job drivers and allocate each job across the provisioned compute resources.

> A task pinned to one context does not demonstrate cross-cloud optimization.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source 4. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Source walkthrough

The normal compute path creates Pods directly

One SkyPilot cluster · num_nodes: 2

Head Pod

v1/Pod · provider-side cluster name + head suffix

Worker Pod

v1/Pod · same cluster grouping label

Created by the Kubernetes provisioner through create_namespaced_pod.

Reuse is part of the path

A matching running Pod can be reused when the named cluster already exists.

One-node clusters still execute work

The head is also a compute node. It is not a dedicated managed-job controller.

> Submitting sky exec does not create a Kubernetes Job for that execution.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Source walkthrough

The application does not need to be a Ray program

Inside a SkyPilot head compute Pod

SKYPILOT CONTROL INSIDE COMPUTE

skylet + cluster job table / queue

Track job states and launch generated job drivers.

EXECUTION RUNTIME

Ray driver + resource placement

Dispatch the supplied shell script to selected compute nodes.

USER WORK

python3, torchrun or another command

The user owns the application algorithm and its outputs.

The Pod can stay Running after the job succeeds because its compute environment remains alive.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source 4. SkyPilot source

Source walkthrough

Files, setup and run have separate destinations

INPUT	OPERATION	DESTINATION OR RESULT
workdir files	Synchronize for launch or exec	Code and working files on the compute
file mounts	Synchronize in the launch preparation path	Configured mounted or copied data
setup script	Prepare the environment	Dependencies available before the job
run script	Execute through the runtime	Application processes and per-job logs

> If exec sends new code that needs a new package, the unchanged setup environment may be insufficient.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Source walkthrough

A pending job waits for compute resources inside the cluster

Cluster job queue
Record submitted work

launch driver →

Generated Ray driver
Request resource bundles / placement group

resources available →

User run script
Execute on the selected nodes

Job drivers are launched in FIFO order. Their resource requirements determine when execution can begin and which jobs can overlap.

> This queue is separate from both the API request queue and managed-job orchestration.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · INSIDE THE PLATFORM

Interactive source walkthrough

Walk one launch from request to completed calculation

01	**Submit cpu-task.yaml**	The CLI sends the task and the cluster name cpu-lab to the API server. The server tracks a long-running launch request.
02	**Choose allowed compute**	The optimizer plans within the requested resources and eligible Kubernetes infrastructure. Kubernetes node placement has not happened yet.
03	**Provision the compute Pod**	The provisioner creates or reuses the head Pod. The Kubernetes scheduler places it and the runtime bootstrap prepares SkyPilot and Ray.
04	**Sync files and prepare the environment**	launch synchronizes workdir and configured mounts, then runs setup. In this task setup checks Python before the job is submitted.
05	**Queue and run the calculation**	The cluster queue launches a generated driver. Ray acquires the resources and runs the Python command. The expected arithmetic result is 16417500000.
06	**Keep compute available or tear it down**	The job completes while the Pod can remain Running. exec submits another run using the existing compute; down terminates that compute.

Source-backed simulation. The teaching task was not run through this sequence in the recorded cluster lab.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source 4. SkyPilot source

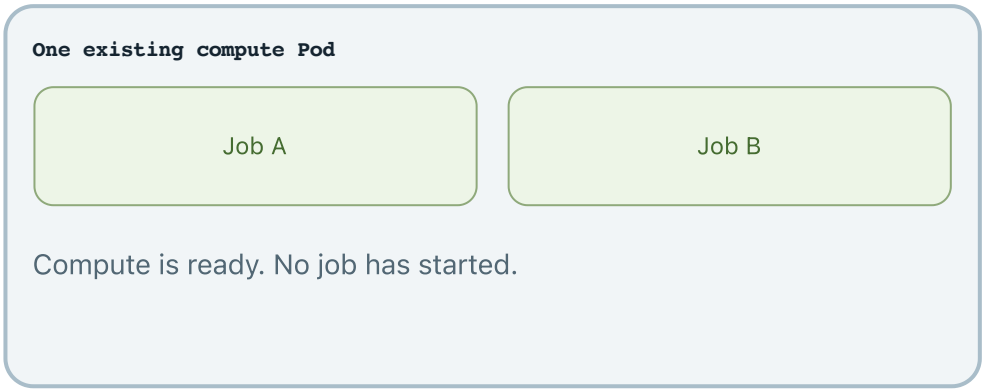
SKYPILOT · JOBS AND COMPUTE

Interactive source walkthrough, not lab execution

One compute Pod can host more than one job

Illustrative jobs · resource requests must fit

```
sky launch task.yaml -c my-cluster
sky exec my-cluster job-a.yaml -d
sky exec my-cluster job-b.yaml -d
```



> Two job associations do not split a single Pod-level utilization sample into two measurements.

Walkthrough: existing compute Pod → Job A starts → Job B starts on the same Pod → two jobs share one Pod-level usage sample. Platform state lists jobs; runtime evidence is needed to split usage.

Sources: 1. SkyPilot source 2. SkyPilot source

SKYPILOT · JOBS AND COMPUTE

Source walkthrough

One distributed job can span several compute Pods

One job · num_nodes: 2

Generated Ray driver requests one bundle per node with STRICT_SPREAD

Dispatch the run script and provide node IPs, count and per-job rank.

Existing SkyPilot compute cluster

Head compute Pod

One part of this job

Worker compute Pod

Another part of this job

Head / worker is a compute role. A distributed application rank belongs to this execution.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · JOBS AND COMPUTE

Source walkthrough

Managed jobs own more of the compute lifecycle

Managed-job CLI

```
sky jobs launch -n cpu-batch cpu-task.yaml
sky jobs queue
sky jobs logs -n cpu-batch
sky jobs cancel -n cpu-batch
```

Ordinary dedicated-cluster path

Provision temporary compute, monitor the run, recover from supported infrastructure failures and clean up afterwards.

Application responsibility remains

Application-error retries are configurable. Recovery does not automatically restore a program to its last instruction; checkpoints still matter.

Sources: 1. SkyPilot source 2. SkyPilot source

SKYPILOT · JOBS AND COMPUTE

Source walkthrough

A remote API server can host the managed-job controllers

New remote API server · consolidation enabled by default

API request handling

Accept and track managed jobs

Controller processes

Provision, monitor, recover and clean up job compute

control operations



Job compute clusters

Job A compute

Head and optional workers

Job B compute

Head and optional workers

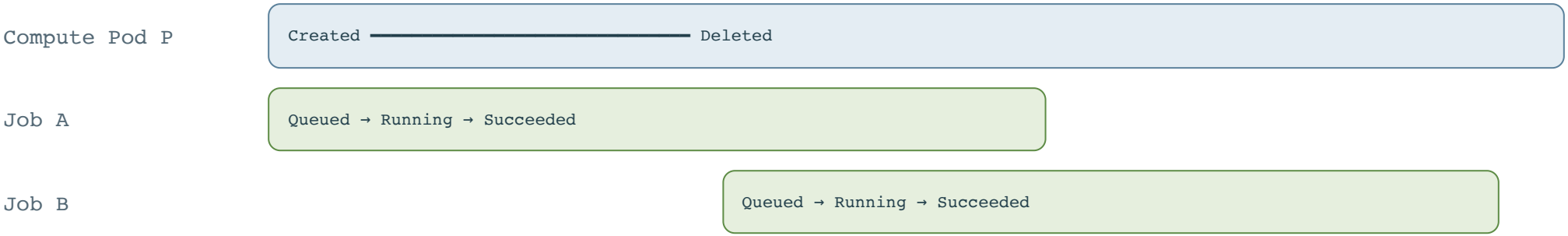
Local API defaults, existing controller clusters and explicit consolidation_mode: false can use a separate controller VM or Pod. A separate controller Pod is not universal.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · EXECUTION IDENTITY

Source walkthrough

A job and its compute can finish at different times



Reusable cluster

Success of job A can leave the same Pod available for job B.

Recovery

A managed job can require replacement compute. Platform job identity and Kubernetes Pod lifetime must remain distinct.

> A Running compute Pod does not prove that Ray initialized or that a user job succeeded.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · EXECUTION IDENTITY

Source walkthrough

Read the metadata at its actual scope

METADATA	MEANING	LIMIT
Label: skypilot-cluster-name	Provider-safe compute cluster grouping	Reusable after deletion; not an incarnation UID
Annotation: skypilot-cluster-name	Original display name	Can differ from the provider-safe label
Label: ray-node-type	head or worker compute role	Not the current application job or rank
Annotations: skypilot-managed-job-*	Creation-time managed-job context	Not ordinary exec job IDs or a live list of pooled jobs

> Scope IDs to the relevant installation. Keep the Pod UID for the actual Kubernetes lifetime.

Sources: 1. SkyPilot source 2. SkyPilot source 3. SkyPilot source

SKYPILOT · EXECUTION IDENTITY

Source behavior + captured metadata

The native component label is a Pod name

CAPTURED FIELD	HEAD	WORKER
ray-node-type	`head`	`worker`
skypilot-head-node	`1`	`0`
component	`...-head`	`...-worker1`
skypilot-cluster-name	Same provider-side cluster name	Same provider-side cluster name
metadata.uid	Head Pod UID	Different worker Pod UID

> Use the explicit compute-role field for a role mapping. Copying component blindly would create one component per Pod name.

Sources: 1. SkyPilot source 2. SkyPilot source 3. Recorded research evidence

SKYPILOT · EXECUTION IDENTITY

Recorded lab boundaries

The recorded SkyPilot lab stopped before successful jobs

Observed

Compute metadata

Head and worker Pod fields, real Pod identities and UID-safe association checks.

Association checks

Recorded metadata joins preserved Pod identity across the cases tested.

Not demonstrated

Runtime execution

Successful concurrent jobs, multinode application execution and managed-job completion.

Recorded compute failure

The captured attempt ended with DeadlineExceeded. It did not establish successful job execution.

> These slides teach source behavior. They do not turn Pod creation into an end-to-end runtime result.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. Recorded research evidence

Source walkthrough

Three views can describe the same work

VIEW	OSMO EXAMPLE	SKYPILOT EXAMPLE
Platform work	Workflow → group → task attempt	Task → cluster job or managed job
Kubernetes observation	Task Pods and scheduling resources	Head / worker compute Pods
Metric grouping	Workflow and task dimensions	Compute cluster and role dimensions

The grouping can be useful
without claiming a new Kubernetes owner.

Sources: 1. OSMO source 2. SkyPilot source 3. Karta exporter KEP

Source walkthrough

Karta preserves the real Kubernetes root

Native controller chain

Deployment

Root UID

owns ReplicaSet

↓

ReplicaSet

Owner reference points upward

owns Pod

↓

Pod

Observed Pod UID

Captured platform task / compute Pod

OSMO or SkyPilot state

External grouping and lifecycle

Pod metadata refers to this platform context

Ownerless Pod

The Pod itself is the Kubernetes root

No ownerReference makes the database record a Kubernetes object.

In the inspected exporter, a Pod root needs a selected Pod Karta such as core-pod-v1.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. Exporter contract

KARTA · WHAT THE METRIC KNOWS

Illustrative native mapping from the definition

Native structure gives the labels a declared meaning

JOBSET EXAMPLE POD	COMPONENT	COMPONENT_INSTANCE	REPLICA_GROUP_ID
Job 0 · Pod A	replicatedjob	worker	0
Job 0 · Pod B	replicatedjob	worker	0
Job 1 · Pod C	replicatedjob	worker	1
Job 1 · Pod D	replicatedjob	worker	1

One root

JobSet UID

One named entry

worker

Two repeated Jobs

Each contains two Pods

> The group is a definition-selected repeated unit. It is not the Pod index or the replica count.

Sources: 1. JobSet Karta definition 2. Metric label contract

KARTA · WHAT THE METRIC KNOWS

Source walkthrough

An info sample is membership, not utilization

Identity path

Karta + configured attributes

Real root, Pod UID and optional metadata

OTel export →

Prometheus info series

karta_pod_workload_info = 1

Usage path

DCGM / resource telemetry

Independent GPU / CPU measurements

scrape →

Prometheus usage series

Measured value with its available identity

> Grafana can group compatible usage and membership observations. DCGM does not automatically supply a Pod UID.

A usage join needs compatible identity and time windows, deduplication and a common cluster label when federated.

Sources: 1. Metric contract and resource-query caveats 2. Recorded research evidence 3. NVIDIA DCGM exporter

Recorded join design and checks

The tested association adds metadata without changing roots

Karta Pod info
namespace + pod_uid + real root

UID-safe join →

KSM metadata
Allowlisted platform labels; uid normalized to pod_uid

record →

Enriched association
Original identity + external workflow or cluster key

OSMO grouping
The tested join attaches the full workflow UUID to each observed Pod.

SkyPilot grouping
Attach compute cluster identity. Current jobs and exclusive job usage still require platform or runtime state.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. Recorded research evidence

Recorded rule and regression cases

Join by Pod UID and reject conflicting metadata

Recorded association rule · single-cluster scope

```
karta_pod_workload_info{observation_valid="true"}
* on (namespace, pod_uid) group_left(label_osmo_workflow_uuid)
(
  research:osmo_pod_metadata
  and on (namespace, pod_uid)
  (count by (namespace, pod_uid) (research:osmo_pod_metadata) == 1)
)
```

Normalize first

The metadata rule renames KSM uid and removes duplicate scrape observations.

One association

Exactly one remaining metadata row must identify the workflow for this Pod.

Keep identity

The resulting sample retains the Karta root and Pod identity fields.

research:* names are recording rules from the experiment. Federation needs a common cluster dimension in every join.

Sources: 1. Recorded research evidence 2. Recorded research evidence

KARTA · WHAT THE METRIC KNOWS

Interactive explanation of recorded UID checks

A reused name must not reuse an old association

One Pod name across time

UID A

The old Pod exists.

Association result

A ↔ A

The identities match.

> After replacement, the old Karta sample must not join the new Pod metadata merely because the name matches.

Schematic A / B identifiers. Real Pod UID replacement and mismatch rejection have recorded evidence.

Walkthrough: UID A matches → old Pod is deleted → the name is reused by UID B → A does not match B → only the B-to-B association is accepted.

Sources: 1. Recorded research evidence 2. Recorded research evidence

GENERIC LABELS · ISSUE 518

Proposal · issue 518 snapshot

Issue 518 proposes a reusable metadata evaluator

Pod or workload root

Full Kubernetes object supplied as object

configured CEL →

String result

Validated custom label value

cache before recording →

OTel attributes

Prometheus and OTLP can export the same enrichment

Ordinary enrichment

Add workflow UUIDs, compute cluster names, teams or roles under custom label names.

Current boundary

The proposal rejects built-in names, including component and submitted_by. Changing that is a contract decision.

> This is a proposed feature. The recorded KSM join uses a separate implementation path.

Sources: 1. Generic CEL metric labels · issue 518

GENERIC LABELS · ISSUE 518

Captured input + proposed evaluation

Show the actual object passed to the expression

object = selected fields of the captured Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  namespace: test-ref
  uid: f799452b-76d1-4d16-abd4-9a63f8e9d3ca
  ownerReferences: []
  labels:
    osmo.task_name: leader
    osmo.workflow_uuid: faa9dfbbaa0b4d009ab7ef312f3d6444
```

CEL expression

```
object.metadata.labels["osmo.task_name"]
```

> "leader"

This expression reads the Pod, not an OSMO workflow record. Production configuration guards missing fields.

leader is this task's authored name. It is not a universal OSMO role.

Sources: 1. Recorded research evidence 2. Issue 518 input object contract

GENERIC LABELS · ISSUE 518

Proposal · not deployed in the lab

A custom task label fits the proposed API

Proposed Helm values from issue 518

```
exporter:
  metricLabels:
    osmo_task_name:
      metrics: [karta_pod_workload_info]
      sources: [pod]
      expression: |
        object.apiVersion == "v1" && object.kind == "Pod" &&
        has(object.metadata.labels) &&
        "osmo.task_name" in object.metadata.labels
          ? object.metadata.labels["osmo.task_name"] : ""
      evaluation:
        events: [create, update]
        mode: continuous
```

> For the previous Pod, the proposed custom attribute is osmo_task_name="leader".

Sources: 1. Proposed metricLabels configuration 2. OSMO source

GENERIC LABELS · ISSUE 518

Proposal

Evaluate on object events; collect cached attributes

Initial discovery or update
Pod / selected root changes

compiled CEL →

Subject-UID cache
Store validated values before datapoint recording

read only →

Collection
Prometheus scrape or OTLP callback

CHANGE	PROPOSED CACHE BEHAVIOR
Selected workload metadata updates	Refresh affected Pod results even without a Pod update
Pod deleted, replaced or leaves scope	Clear the old UID entry
Root association or configuration changes	Invalidate and recompute
Exporter restarts	Reevaluate; the cache is in memory

> No CEL evaluation or Kubernetes API calls during metric collection.

Sources: 1. Issue 518 evaluation and cache lifecycle

GENERIC LABELS · ISSUE 518

Proposal

Use continuous values for metadata that can change

MODE	WHAT IT DOES	IMPORTANT CONSEQUENCE
continuous	Replace the value on selected events	Remove the label when no selected source produces a value
untilValue	Keep the first nonempty result	A Pod fallback stays fixed even if workload metadata appears later

`sources: [workload, pod]` Try the workload first. Empty value or evaluation error tries the next source.

Defaults are create/update events and continuous mode. Runtime errors are reported while the base metric survives.

Sources: 1. Issue 518 modes, defaults and source priority

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate contract extension

The same CEL machinery can support component mappings

Yes: use an explicit mapping contract for the three attribution fields.

OUTPUT	CANDIDATE PLATFORM INTERPRETATION	STILL REQUIRED
component	A declared task or compute role	A consistent meaning for the deployment
component_instance	A named entry within that component	Real metadata or a declared empty value
replica_group_id	A declared repeated application unit	An application convention; no guess from a Pod index

> Keep ordinary custom labels as proposed. Make built-in attribution overrides a separate, explicit opt-in.

Sources: 1. Current built-in rejection boundary 2. Native attribution semantics

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Captured baseline → candidate mapping

OSMO can expose the task in a familiar tuple

METRIC FIELD	CAPTURED BASE VALUE	CANDIDATE MAPPED VALUE
workload_kind	Pod	Pod
workload_uid	This Pod UID	Same Pod UID
component	pod	task
component_instance	empty	leader from osmo.task_name
replica_group_id	empty	empty
External dimensions	absent from the base row	Full workflow, group and task UUIDs; retry ID

The sibling task becomes task / worker / empty. The external workflow key groups both Pod roots.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. OSMO source

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate mapping conventions

A stronger OSMO group mapping needs a declared convention

Conservative task view

task / task name / empty

Use the actual authored task name. Preserve full task UUID for identity because display names can be shortened.

`lead_container=true` identifies a lead. Its absence does not mean worker.

Declared repeated application unit

role / instance / group UUID

If the application defines each group as one repeated unit, group UUID can supply that group key and controlled metadata can supply the role.

Different roles inside one repeated unit are possible.

> Group heterogeneity does not forbid a mapping. The missing piece is a declared meaning. Retry ID is an attempt, not a replica group.

Sources: 1. OSMO source 2. OSMO source 3. Karta LWS and group semantics

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate mapping from captured roles

SkyPilot already supplies explicit compute roles

FIELD	HEAD CANDIDATE	WORKER CANDIDATE
component	head	worker
component_instance	empty	empty
replica_group_id	empty	empty
External grouping	skypilot_cluster_name	Same skypilot_cluster_name
Kubernetes root	Head Pod UID	Worker Pod UID

Read `ray-node-type` only when it is head or worker and the platform selector matches.

> This convention describes compute structure. It does not claim application-job attribution.

Sources: 1. SkyPilot source 2. SkyPilot source 3. Recorded research evidence

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate precedence contract

Native definitions keep priority over metadata mapping

Does Karta have a more specific native definition?

Yes

Use native structure

JobSet, a custom Pod Karta or another selected mapping

No

Check the narrow fallback scope

Verified ownerless Pod root + selected generic core-pod-v1 definition

One matching configured platform rule may replace its generic attribution tuple.

> Eligibility comes from verified ownership and definition selection, not just component="pod" or a platform label.

Sources: 1. Existing attribution contract 2. Recorded research evidence 3. Issue 518 boundary to extend

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate atomic mapping contract

Publish the three attribution fields as one result

component	component_instance	replica_group_id

CASE	CANDIDATE BEHAVIOR
Exactly one valid rule	Publish one coherent tuple and one Pod row
Intentionally empty instance or group	Accept the declared empty value
Ambiguous rules, missing metadata or evaluation error	Remove the mapped tuple, retain verified base attribution and report a bounded diagnostic
Value changes or Pod UID changes	Reevaluate the tuple; never mix frozen and current fields

> Expose mapping provenance so a panel can distinguish native structure from configured metadata.

Sources: 1. Base metric and validity contract 2. Issue 518 cache and error behavior

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Preserved contract for the candidate extension

Keep identity, status and trust outside the override

Identity and ownership

Pod UID, root UID / GVK, namespace, names and owner_resolution remain Karta observations.

Validity and submitter

A label expression cannot turn an invalid observation valid or replace authenticated submitted_by.

Status

For these bare Pods, workload status remains Pod status. It is not OSMO workflow status or SkyPilot job status.

> A platform label is a selection input. It does not authenticate the actor who wrote it.

Sources: 1. Metric identity, status and validity 2. Issue 518 preserves submitter contract

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate output, not a captured exporter row

The enriched row keeps both identities visible

Candidate enriched sample · schematic values, selected labels

```
karta_pod_workload_info{
  namespace="test-ref", pod_name="task-pod", pod_uid="P",
  workload_kind="Pod", workload_uid="P",
  component="task", component_instance="leader", replica_group_id="",
  observation_valid="true",
  platform_instance="osmo-lab", osmo_workflow_uuid="W",
  osmo_task_uuid="T", osmo_retry_id="0"
} 1
```

P

Real Kubernetes Pod identity

W + T

External workflow and logical task

> Changing component does not merge Pod roots. A query grouped by workload_uid still returns each Pod separately.

Sources: 1. Recorded research evidence 2. Metric contract 3. Issue 518 enrichment proposal

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate query, not executed in this lab

Group by the external key when the question is the workflow

Candidate query · count mapped Pods per workflow and task

```
count by (
  platform_instance, namespace, osmo_workflow_uuid,
  component, component_instance, osmo_task_uuid
) (
  karta_pod_workload_info{
    observation_valid="true", osmo_workflow_uuid!=""
  }
)
```

For one leader Pod and one worker Pod in workflow W, the result is one Pod in each task group.

Assumes one observation per Pod and nonconflicting mappings. Add a common Kubernetes cluster label for federation. Use task UUID to distinguish colliding shortened display names.

> This query counts membership. A CPU or GPU usage query needs a separate, compatible resource measurement.

Sources: 1. Info metric semantics 2. Recorded research evidence 3. Proposed custom label dimensions

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Candidate output and query

The compute view groups SkyPilot Pods by cluster and role

Candidate head row · schematic IDs

```
karta_pod_workload_info{
  namespace="team", pod_uid="H", workload_uid="H",
  workload_kind="Pod", component="head",
  component_instance="", replica_group_id="",
  observation_valid="true",
  skypilot_cluster_name="cpu-lab-x1"
} 1
```

Candidate compute Pod count

```
count by (
  namespace, skypilot_cluster_name, component
) (
  karta_pod_workload_info{
    observation_valid="true",
    skypilot_cluster_name!=""
  }
)
```

A worker row keeps its own Pod UID and uses component="worker". The same cluster key groups the compute.

Single-cluster query with one canonical observation per Pod. Federation adds a common Kubernetes cluster label. Scope platform correlation by installation; the cluster name is reusable, not an incarnation ID.

Sources: 1. SkyPilot source 2. Info metric and native identity 3. Issue 518 extension boundary

STRUCTURAL MAPPING · CANDIDATE EXTENSION

Illustrative arithmetic from the shared-compute model

A shared Pod must contribute usage once

Illustrative measurement

8 CPU cores

Pod P is running jobs A and B.

Compute-cluster total	8
Job A exclusive usage	?
Job B exclusive usage	?

> Copying the 8-core sample onto both job rows and summing produces 16. That is duplication, not attribution.

Keep one canonical Pod mapping. A separate active-job relation can list jobs. Exclusive usage needs runtime measurements or an explicitly labeled allocation model.

Sources: 1. SkyPilot source 2. SkyPilot source 3. Recorded research evidence

INTEGRATION · CHOOSE THE REQUIRED DEPTH

Reference-branch capability + integration gap

A reference can fetch a real related Kubernetes object

Input Pod
pod-group-name annotation

configured GVK + name →

CEL reference lookup
Read the same-namespace PodGroup

object + UID →

Evaluation context
Use fields from that actual object

What this adds

A declared relationship to an existing Kubernetes resource with its own real UID.

What it does not add

A controller owner relationship, an OSMO database query or the current SkyPilot job list.

> The reference branch still needs exporter watches, invalidation and a public relation output to integrate this path.

Sources: 1. Recorded research evidence 2. CEL reference branch

INTEGRATION · CHOOSE THE REQUIRED DEPTH

Longer-term integration proposal

Authoritative platform status needs a lifecycle source

Platform state

Workflow / cluster / current job records

maintained bridge →

Queryable projection

Explicit identity, status, scope and cleanup

declared association →

Observability

Platform status alongside Pod execution

Why labels are insufficient

An OSMO workflow can have waiting tasks with no Pods. A SkyPilot cluster can have queued or completed jobs while its Pod stays Running.

What the bridge owns

Create, update and delete semantics, tenancy, completeness and external identity. A real Kubernetes projection gets its own actual UID.

> OSMO's optional per-group ConfigMap is a group projection, not a full workflow lifecycle.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. SkyPilot source

INTEGRATION · CHOOSE THE REQUIRED DEPTH

Design comparison

Choose the option that answers the product question

NEED	USEFUL OPTION	ADDITIONAL CONTRACT OR WORK
Workflow or compute-cluster grouping	Tested KSM joins, or proposed generic labels	External identity dimensions, UID handling, conflict policy
Shared component-oriented panels	Guarded attribution mapping	Opt-in, atomic tuple, native precedence, provenance and lifecycle tests
Authoritative platform status	Platform state adapter or maintained object projection	Lifecycle, completeness, tenancy and deletion
Per-job usage on shared compute	Runtime measurement or declared allocation model	Job/resource/time association; no duplicate accounting

> Generic CEL makes extraction reusable. It does not remove the need to define the meaning of the result.

Sources: 1. Issue 518 2. Recorded research evidence 3. Recorded research evidence 4. SkyPilot source

INTEGRATION · CHOOSE THE REQUIRED DEPTH

Candidate verification plan

The extension needs tests at the semantic boundaries

BOUNDARY	REQUIRED RESULT
Native JobSet or custom Pod definition	Keep native attribution unchanged
Eligible generic Pod root	Apply one declared platform tuple and preserve root identity
Missing, conflicting or changing metadata	No partial tuple; verified base row survives with diagnostics
Deletion, new UID, reparenting or config change	Clear stale mapping state
Invalid base observation or shared jobs	No invented validity and no duplicate canonical Pod row
Prometheus and OTLP collection	Same cached attributes; no CEL or API work during collection

Sources: 1. Issue 518 acceptance criteria 2. Existing root / validity contract

INTEGRATION · CHOOSE THE REQUIRED DEPTH

Evidence and remaining gaps

The evidence supports a narrower claim than full runtime integration

EVIDENCE	SUPPORTED CONCLUSION
Completed OSMO CPU single task and group	Real workflow/task labels and bare-Pod identity are available
Captured SkyPilot head and worker Pods	Compute roles and grouping metadata are available
Recorded Karta / KSM joins and UID cases	Metadata association can preserve Kubernetes roots
Source inspection	Execution paths, retries, sharing and controller placement are explained
Generic labels and guarded overrides	Proposed designs; no implemented override result claimed

Remaining runtime gaps: SkyPilot job execution, concurrent jobs and managed jobs; OSMO live retry. CPU identity tests are not GPU utilization or application-efficiency measurements.

Sources: 1. Recorded research evidence 2. Recorded research evidence 3. Issue 518 proposal

INTEGRATION · CHOOSE THE REQUIRED DEPTH

Proposed integration direction

A workload view you can trace back to each Pod

OSMO	Workflow → named tasks → attempts → observed Pods.
SkyPilot	Compute cluster → roles → jobs with their own lifetimes.
Karta	Real roots + explicit external grouping + declared attribution.

Use generic labels for grouping. Add guarded structural mapping when common component panels need it. Count each Pod once.

Sources: 1. Issue 518 2. Exporter contract 3. Recorded research evidence